

Laboratorio evolutivo 06

Operación y automatización de Digital Café Luna en AWS

Cloud Computing Posgrado · Semana 6 · Cuenta AWS estándar

Duración	Modalidad	Entorno	Carácter
80–90 min	Equipos de 4	Cuenta AWS estándar	Formativo; con evidencia

1. Propósito y resultado

Demostrar que una arquitectura cloud debe ser observable y repetible, no solamente existir. El equipo desplegará una unidad declarativa con CloudFormation, observará una señal causada de forma controlada, realizará un cambio no destructivo y eliminará el entorno completo mediante el mismo stack.

Al finalizar, el equipo podrá reconstruir la cadena: definición declarativa → stack → EC2 → carga local → CPUUtilization → condición → alarma → evidencia → update → cleanup declarativo.

Hipótesis práctica

Una infraestructura definida como código puede desplegarse, observarse, modificarse y eliminarse como una unidad reproducible.

Objetivos de aprendizaje

- Distinguir template válido, despliegue exitoso y comportamiento observable.
- Mapear recursos lógicos de CloudFormation con recursos físicos creados en AWS.
- Relacionar una carga CPU controlada con datapoints de CPUUtilization y con el estado de una alarma.
- Ejecutar un update declarativo sin reemplazar innecesariamente la instancia.
- Eliminar como stack lo que se creó como stack y restaurar el baseline S01.

Ruta de trabajo

Tiempo	Bloque	Salida
0–10 min	Preflight + lectura del template	Baseline y recursos lógicos identificados
10–25 min	Validación + create stack	CREATE_COMPLETE
25–45 min	EC2 + CPUUtilization	Datapoints y elevación visibles
45–60 min	CloudWatch Alarm + causalidad	INSUFFICIENT_DATA → ALARM
60–70 min	Update v1 → v2	UPDATE_COMPLETE sin replacement
70–80 min	Troubleshooting + evidencias	Cadena causal explicada
80–90 min	Delete stack + cierre	Baseline S01 restaurado

Arquitectura objetivo

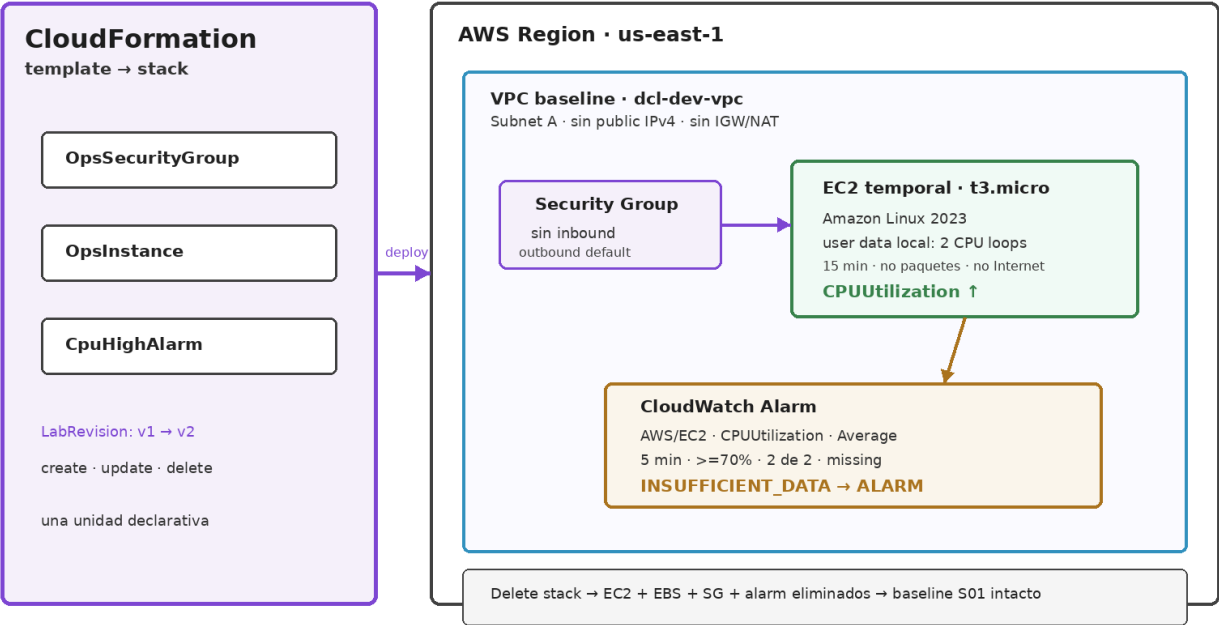


Figura 1. Una unidad declarativa produce un recurso temporal, una señal observable y una alarma; el cleanup también se ejecuta como stack.

2. Baseline y preflight

Qué hacemos: Confirmar Región, baseline S01 y permisos normales de la cuenta antes de desplegar.

Por qué: El stack depende de recursos persistentes que ya existen, pero no debe recrearlos ni modificar su conectividad. La EC2 no necesita Internet, public IPv4, SSH ni access keys.

Elemento	Valor esperado
Región	us-east-1
VPC	dcl-dev-vpc · 10.20.0.0/16
Subnet	dcl-dev-subnet-a · 10.20.1.0/24
Conectividad EC2	Sin public IPv4 · sin IGW/NAT requeridos
Recursos temporales	SG + EC2 + CloudWatch Alarm dentro de un stack

1. Confirman que VPC y Subnet A pertenecen a us-east-1 y registren sus IDs.
2. Confirman que no existe un stack dcl-dev-stack-ops-s06 activo de una ejecución anterior.
3. La identidad de la consola debe poder usar CloudFormation, EC2, CloudWatch y leer el parámetro público de SSM. No creen access keys como workaround.
4. Si una policy IAM/SCP o una cuota bloquea el despliegue, registren el evento del stack antes de cambiar la configuración.

Validación: El baseline coincide con S01 y el equipo dispone de los IDs de VPC/Subnet. No se introduce conectividad adicional.

3. Template de trabajo — parte 1

El template es deliberadamente pequeño: recibe el VPC/Subnet existentes como parámetros, resuelve Amazon Linux 2023 mediante un parámetro público de Systems Manager y crea únicamente tres recursos administrados por el stack.

```
AWSTemplateFormatVersion: '2010-09-09'
Description: S06 - Operacion y automatizacion - Digital Cafe Luna

Parameters:
  VpcId:
    Type: AWS::EC2::VPC::Id
    Description: VPC baseline dcl-dev-vpc
  SubnetId:
    Type: AWS::EC2::Subnet::Id
    Description: Subnet A baseline dcl-dev-subnet-a
  AmiId:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
  CpuThreshold:
    Type: Number
    Default: 70
  LabRevision:
    Type: String
    Default: v1
    AllowedValues: [v1, v2]

Resources:
  OpsSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: S06 - no inbound access
      VpcId: !Ref VpcId
      Tags:
        - Key: Name
          Value: !Sub '${AWS::StackName}-sg'
        - Key: dcl:project
          Value: digital-cafe-luna
        - Key: dcl:lab-revision
          Value: !Ref LabRevision

  OpsInstance:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref AmiId
      InstanceType: t3.micro
      Monitoring: false
      MetadataOptions:
        HttpTokens: required
        HttpEndpoint: enabled
      NetworkInterfaces:
        - DeviceIndex: '0'
          AssociatePublicIpAddress: false
          DeleteOnTermination: true
          SubnetId: !Ref SubnetId
          GroupSet:
            - !Ref OpsSecurityGroup
      BlockDeviceMappings:
        - DeviceName: /dev/xvda
          Ebs:
            VolumeType: gp3
            VolumeSize: 8
            Encrypted: true
            DeleteOnTermination: true
      UserData:
        Fn::Base64: !Sub |
          #!/bin/bash
```

Template de trabajo — parte 2

```
set -euxo pipefail
LOG=/var/tmp/dcl-s06-cpu-load.log
echo "$(date -ls) start" > "$LOG"
for i in 1 2; do
  timeout 900 bash -c 'while ;; do ;; done' &
done
wait
echo "$(date -ls) end" >> "$LOG"
```

Tags:

- Key: Name
Value: !Sub '\${AWS::StackName}-ec2'
- Key: dcl:project
Value: digital-cafe-luna
- Key: dcl:lab-revision
Value: !Ref LabRevision

CpuHighAlarm:

Type: AWS::CloudWatch::Alarm

Properties:

AlarmName: !Sub '\${AWS::StackName}-cpu-high'
AlarmDescription: !Sub 'S06 \${LabRevision}: CPU controlada >= \${CpuThreshold}%'

ActionsEnabled: false

Namespace: AWS/EC2

MetricName: CPUUtilization

Dimensions:

- Name: InstanceId
Value: !Ref OpsInstance

Statistic: Average

Period: 300

EvaluationPeriods: 2

DatapointsToAlarm: 2

Threshold: !Ref CpuThreshold

ComparisonOperator: GreaterThanOrEqualToThreshold

TreatMissingData: missing

Tags:

- Key: dcl:project
Value: digital-cafe-luna
- Key: dcl:lab-revision
Value: !Ref LabRevision

Outputs:

InstanceId:

Value: !Ref OpsInstance

AlarmName:

Value: !Ref CpuHighAlarm

Revision:

Value: !Ref LabRevision

OJO TÉCNICO: Monitoring: false conserva basic monitoring. CPUUtilization se publica en períodos de 5 minutos; la alarma usa Period=300, EvaluationPeriods=2 y DatapointsToAlarm=2. El user data mantiene dos procesos CPU-bound durante 900 s y termina solo.

Prueba 1 — validar IaC antes de desplegar

Qué hacemos: Revisar la estructura del template y ejecutar una validación sin crear recursos.

Por qué: template válido ≠ infraestructura desplegada. La lectura debe permitir reconocer parámetros, referencias y dependencias sin convertir S06 en un taller de YAML.

5. Guarden el bloque anterior como s06-ops-cloudformation.yaml.
6. En CloudShell pueden cargar el archivo y ejecutar el comando siguiente. CloudShell usa la sesión autenticada; no creen access keys.
7. Identifiquen VpcId, SubnetId, Amild, CpuThreshold y LabRevision; luego identifiquen OpsSecurityGroup, OpsInstance y CpuHighAlarm.

```
aws cloudformation validate-template \  
--template-body file://s06-ops-cloudformation.yaml
```

Validación: El comando devuelve Description/Parameters sin ValidationError. Todavía no existe stack ni infraestructura S06.

4. Prueba 2 — crear el stack

Qué hacemos: Crear dcl-dev-stack-ops-s06 usando el template validado y los IDs del baseline.

Por qué: CloudFormation convierte definiciones lógicas en recursos físicos y conserva el estado del despliegue como una unidad administrada.

- En CloudFormation > Stacks, seleccionen Create stack > With new resources y carguen s06-ops-cloudformation.yaml.
- Stack name = dcl-dev-stack-ops-s06.
- VpcId = ID de dcl-dev-vpc; SubnetId = ID de dcl-dev-subnet-a; CpuThreshold = 70; LabRevision = v1. Mantengan el Amild por defecto.
- No agreguen IAM capabilities: el template no crea recursos IAM.
- Crean el stack y sigan la pestaña Events hasta CREATE_COMPLETE.

CREATE_COMPLETE demuestra que CloudFormation terminó el despliegue. No demuestra que la EC2 esté produciendo la señal esperada ni que la alarma haya evaluado la condición.

Logical resource → physical resource

Logical ID	Qué debe mapearse
OpsSecurityGroup	Security Group físico creado en dcl-dev-vpc
OpsInstance	Instanceld de la EC2 temporal en Subnet A
CpuHighAlarm	Nombre físico de la alarma del stack

Validación: Stack = CREATE_COMPLETE; EC2 = running; la instancia no tiene public IPv4; el SG no tiene inbound; la pestaña Resources permite mapear cada logical ID con su physical ID.

Evidencia: captura de CREATE_COMPLETE + tabla/registro breve con los tres mappings. Eviten capturas redundantes de cada recurso por separado.

Ventana de observación

El experimento usa basic monitoring: CPUUtilization se publica en períodos de 5 minutos. La carga dura 15 minutos para cubrir al menos dos períodos completos aun si el primer datapoint es parcial. Reserven 25–35 minutos desde el inicio del stack para observar métrica y transición de alarma; el tiempo exacto de publicación no está garantizado.

5. Prueba 3 — observar CPUUtilization

Qué hacemos: Buscar la métrica CPUUtilization de la EC2 creada por el stack y relacionar su elevación con el user data.

Por qué: Una métrica solo es evidencia cuando podemos explicar qué comportamiento la produjo.

- Copien el Instanceld desde CloudFormation > Resources.
- Abrir CloudWatch > Metrics > All metrics > EC2 > Per-Instance Metrics.
- Filtren por Instanceld y seleccionen CPUUtilization.
- Usen Statistic = Average y Period = 5 minutes; ajusten la ventana temporal para incluir el arranque de la instancia.
- Esperen a que aparezcan datapoints y observen una elevación claramente asociada al intervalo de carga.

Validación: La gráfica muestra datapoints de CPUUtilization y un tramo elevado coherente con la carga CPU-bound iniciada por user data.

Evidencia: una sola gráfica donde sean legibles Instanceld, métrica, período y ventana temporal. No basta con mostrar que CPUUtilization existe.

Prueba 4 — observar la alarma

Qué hacemos: Abrir la alarma creada por el stack y observar cómo evalúa la métrica anterior.

Por qué: Una alarma expresa una condición sobre una señal; no diagnostica por sí sola la salud de la instancia.

Propiedad	Configuración exacta
Métrica	AWS/EC2 · CPUUtilization
Dimension	InstanceId = OpsInstance
Statistic	Average
Period	300 s / 5 min
Threshold	>= 70%
EvaluationPeriods	2
DatapointsToAlarm	2
TreatMissingData	missing
ActionsEnabled	false

ALARM no significa automáticamente que la instancia esté dañada. Significa que, con esta configuración, suficientes datapoints hicieron verdadera la condición $\geq 70\%$.

Validación: La alarma parte típicamente en INSUFFICIENT_DATA y, con dos períodos breaching disponibles, puede pasar a ALARM. No es obligatorio esperar el retorno a OK.

Evidencia: detalle de la alarma con gráfica, threshold, estado y timestamp/períodos suficientes para relacionarlo con la carga.

6. Prueba 5 — explicar causalidad

La prueba central de S06 no es “ver una alarma roja”. El equipo debe reconstruir una cadena causal respaldada por evidencias independientes.

Eslabón	Evidencia observable
user data	Template contiene dos procesos CPU-bound con timeout de 900 s
carga CPU	Intervalo de CPUUtilization elevado en la instancia correcta
datapoints	Métrica AWS/EC2 con período de 5 min
threshold	Alarma configurada con Average $\geq 70\%$
evaluación	2 datapoints de 2 períodos requeridos
estado	Timestamp de transición/estado de la alarma coherente con la carga

Pregunta central: **¿Qué evidencia permite afirmar que la alarma fue causada por la carga controlada y no simplemente porque la alarma existe?**

Prueba 6 — pequeño cambio IaC

Qué hacemos: Actualizar LabRevision de v1 a v2 usando el mismo template.

Por qué: Queremos demostrar estado deseado v1 → cambio declarativo → estado deseado v2 sin provocar replacement deliberado de EC2.

18. Antes del update, registren el InstanceId físico de OpsInstance.
19. En CloudFormation, seleccionen Update stack > Use current template.
20. Cambien únicamente LabRevision de v1 a v2 y revisen el resumen de cambios.
21. Ejecuten el update y esperen UPDATE_COMPLETE.
22. Confirмен que el InstanceId de OpsInstance es el mismo y que el tag dcl:lab-revision refleja v2; la descripción/tag de la alarma también queda actualizado.

Validación: UPDATE_COMPLETE y mismo physical ID para OpsInstance. El cambio afecta metadata/tags y no necesita reemplazar la instancia.

Evidencia: UPDATE_COMPLETE + comparación del InstanceId antes/después + Revision=v2 en Outputs o tags.

7. Troubleshooting basado en evidencia

Síntoma	Qué revisar primero
Stack falla al crear	Events: template · permisos · parámetros · quota · configuración del recurso
EC2 existe pero no aparece métrica	InstanceId · namespace · dimension · período · espera del primer datapoint
Alarm = INSUFFICIENT_DATA	Existencia de datapoints · Period=300 · EvaluationPeriods=2 · dimensión correcta
Métrica supera threshold pero alarma no cambia	Statistic · threshold · datapoints/evaluation periods · tiempo de evaluación
Update intenta replacement	Propiedad modificada · change summary · physical ID esperado
Delete stack falla	Stack Events · recurso bloqueante · dependencia que impide eliminación

Pregunta obligatoria: **¿Qué evidencia permite distinguir un problema de despliegue de un problema de observación?**

8. Evidencias mínimas

- Template IaC utilizado y validado antes del despliegue.
- Stack dcl-dev-stack-ops-s06 en CREATE_COMPLETE.
- Relación logical resource → physical resource para SG, EC2 y alarm.
- Gráfica de CPUUtilization con elevación atribuible a la carga controlada.
- Estado observable de CloudWatch Alarm y su configuración exacta.
- Explicación carga → métrica → threshold → evaluación → alarma.
- Stack update en UPDATE_COMPLETE sin replacement de OpsInstance.
- Delete stack completado y baseline S01 restaurado.

Regla de evidencia: **una captura de CREATE_COMPLETE no demuestra que la arquitectura se comporte como esperamos.**

Criterios de aceptación

- La EC2 no tiene public IPv4, no requiere Internet y no expone inbound.
- El user data no instala paquetes y termina automáticamente después de 15 min.
- CPUUtilization y alarm corresponden al mismo InstanceId.
- El equipo explica la causalidad con timestamps/períodos, no por coincidencia visual.
- El update v1→v2 conserva el physical ID de la EC2.
- El cleanup se realiza con Delete stack y no eliminando recursos manualmente uno por uno.

9. Cost awareness y cleanup declarativo

El lab usa una sola EC2 pequeña, su root EBS, basic EC2 metrics y una alarma temporal. No se crean ALB, NAT Gateway, public IPv4, SNS, CloudWatch Agent, logs adicionales ni detailed monitoring. CloudFormation no es una razón para mantener recursos activos después de obtener la evidencia.

Recurso temporal	Decisión
EC2 t3.micro	Usar solo durante la ventana del lab; terminar mediante Delete stack
Root EBS 8 GiB gp3	DeleteOnTermination=true; se elimina junto con EC2

Recurso temporal	Decisión
Security Group	Sin inbound; eliminado por el stack
CloudWatch Alarm	Temporal; sin acciones/SNS; eliminado por el stack
CloudFormation stack	Unidad de create/update/delete; no persistir después del lab

CONSERVAR COMO BASELINE

- VPC dcl-dev-vpc · 10.20.0.0/16.
- dcl-dev-subnet-a · 10.20.1.0/24 y dcl-dev-subnet-b · 10.20.2.0/24 con sus dos AZ.
- Naming/tags persistentes del curso y evidencias del equipo.

ELIMINAR — cleanup obligatorio

23. En CloudFormation, seleccionar dcl-dev-stack-ops-s06 > Delete.
24. Esperar a que el stack desaparezca de la vista de stacks activos.
25. Verificar que OpsInstance terminó y que su root EBS ya no existe.
26. Verificar que OpsSecurityGroup y CpuHighAlarm fueron eliminados por CloudFormation.
27. No eliminar manualmente los recursos uno por uno salvo troubleshooting documentado de un DELETE_FAILED.
28. Confirmar que VPC y ambas subnets del baseline S01 permanecen intactas.

Validación de cleanup: no quedan EC2, root EBS, SG ni alarm del stack S06. El baseline S01 permanece y no se creó conectividad pública.

10. Cierre técnico

- deployed ≠ operable: CREATE_COMPLETE no sustituye evidencia de comportamiento.
- Una métrica adquiere valor cuando se conecta con una hipótesis y una causa observable.
- Una alarma declara una condición; no explica automáticamente la causa ni la salud del sistema.
- IaC permite tratar create, update y delete como operaciones sobre un estado deseado reproducible.

Fuentes oficiales

- [AWS — Reference latest AMIs using SSM public parameters](#)
- [AWS — Monitor EC2 instances using CloudWatch](#)
- [AWS — CloudWatch alarm evaluation](#)
- [AWS — AWS::EC2::Instance CloudFormation reference](#)
- [AWS — CloudFormation stack update behavior](#)